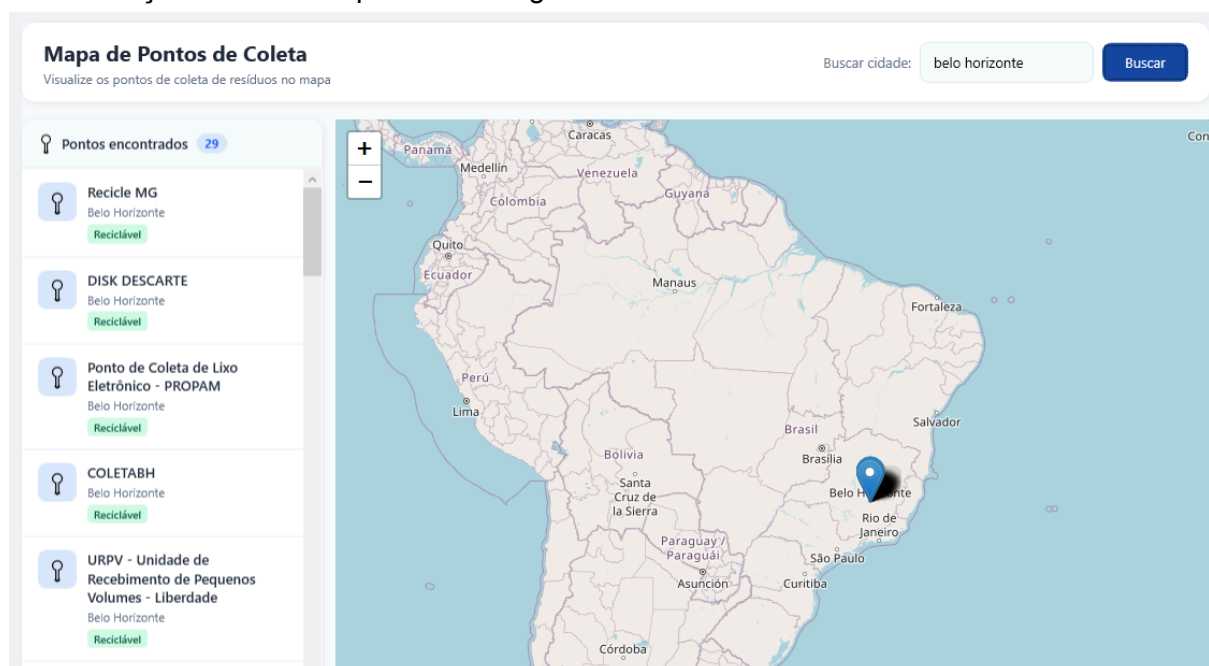


DOCUMENTAÇÃO TÉCNICA: INTEROPERABILIDADE E GESTÃO DO PROJETO REGRAFIK

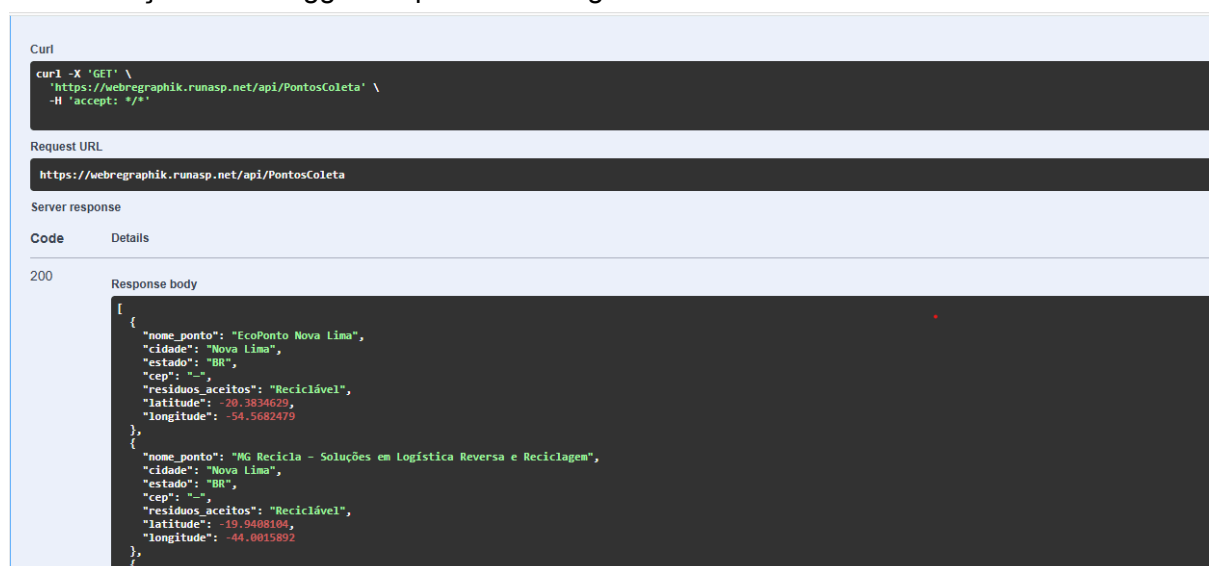
1. Evidências de Interoperabilidade entre Sistemas

A interoperabilidade do sistema **ReGraphik** foi implementada e validada garantindo que duas aplicações distintas (o cliente Desktop WPF e a API REST / Firebase Database) troquem dados de forma síncrona, assíncrona e sem acoplamento rígido.

Sincronização do WPF + Api online Google



Sincronização do Swagger + Api online Google



1.1. Contrato de Comunicação e Formato de Dados (Sintática)

A comunicação baseia-se no protocolo **HTTP/HTTPS**, utilizando o formato universal **JSON (JavaScript Object Notation)** para a serialização e desserialização de objetos. Isso garante que qualquer sistema terceiro possa consumir ou enviar dados para a nossa aplicação, desde que respeite o contrato estabelecido.

- **Exemplo de Payload do Contrato (JSON de Resíduo):**

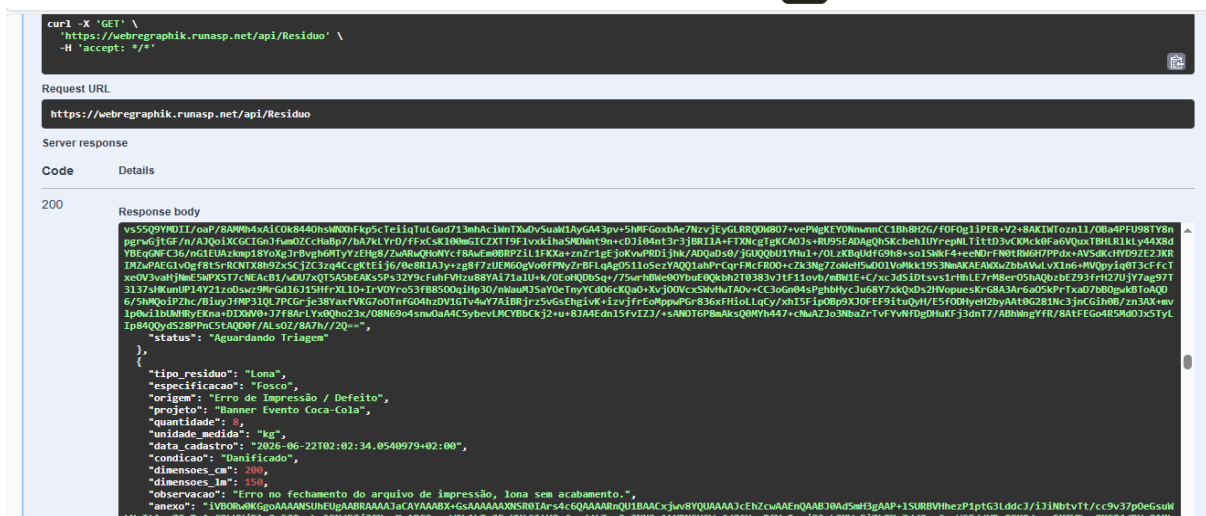
```
{
  "tipo_residuo": "string",
  "especificacao": "string",
  "origem": "string",
  "projeto": "string",
  "quantidade": 0,
  "unidade_medida": "string",
  "data_cadastro": "2026-06-29T20:33:01.095Z",
  "condicao": "string",
  "dimensoes_cm": 0,
  "dimensoes_lm": 0,
  "observacao": "string",
  "anexo": "string",
  "status": "string"
}
```

POST/GET

The screenshot shows a REST client interface with a blue 'Execute' button and a 'Clear' button. Below the buttons is a 'Responses' section. The 'Curl' section displays the following command:

```
curl -X 'POST' \
  'https://webregraphik.runasp.net/api/Residuo' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "tipo_residuo": "Placa PS",
    "especificacao": "Padrão/Comum",
    "origem": "Produção",
    "projeto": "Projeto Novo",
    "quantidade": 2,
    "unidade_medida": "M",
    "data_cadastro": "2026-06-29T20:23:08.364Z",
    "condicao": "bom",
    "dimensoes_cm": 1,
    "dimensoes_lm": 6,
    "observacao": "string",
    "anexo": "string",
    "status": "Disponível"
  }'
```

The 'Request URL' section shows the URL: `https://webregraphik.runasp.net/api/Residuo`



1.2. Arquitetura de Integração (Semântica)

O ecossistema consome múltiplos endpoints para otimização de processos:

1. **Leitura Dinâmica (Fim do Hardcode):** O componente **ComboBox** (Dropdown) de Resíduos Alvo não possui dados estáticos na interface. Ele realiza uma chamada assíncrona (**GetAsync**) para a API/Firebase, mapeia os dados dinamicamente em memória e preenche a UI em tempo de execução.
2. **Desacoplamento via MVVM:** A camada de visualização (WPF) não conhece as regras de conexão com o banco de dados. Ela delega a responsabilidade para a **ViewModel**, que utiliza a biblioteca **HttpClient** e **FirebaseClient** para transporte dos dados.

```

/// <summary>
/// Esta classe é responsável por lidar com a lógica de autorização do usuário, como login, cadastro e validação de tokens de segurança.
/// </summary>

public class AutorizarService : IAutorizarService
{
    private static readonly HttpClient _httpClient = new HttpClient { BaseAddress = new Uri("https://webregraphik.runasp.net/api/") };
    private readonly JsonSerializerOptions _jsonOptions = new JsonSerializerOptions { PropertyNameCaseInsensitive = true };

    public async Task<bool> SolicitarAcessoAsync(string email)
    {
        var content = new StringContent(JsonSerializer.Serialize(new { email }), Encoding.UTF8, "application/json");
        var response = await _httpClient.PostAsync("usuario", content);
        return response.IsSuccessStatusCode;
    }
}

using System.Threading.Tasks;
using Firebase.Database; // + ADICIONAR esta linha

namespace ReGraphik.Services
{
    /// <summary>
    /// Fornece acesso singleton ao FirebaseClient configurado
    /// para o Realtime Database do ReGraphik.
    /// </summary>
    /// <summary>
    /// </summary>
    public static class FirebaseConfig
    {
        // URL do Realtime Database do projeto Firebase
        private const string DatabaseUrl =
            "https://regraphikfirebase-default-rtdb.firebaseio.com/";

        private static FirebaseClient? _client;

        /// <summary>
        /// Instancia unica do FirebaseClient reutilizada por toda a aplicacao.
        /// </summary>
        // referências
        public static FirebaseClient Client =>
            _client ??= new FirebaseClient(DatabaseUrl);
    }
}

```

1.3. Resiliência e Tratamento de Erros de Conexão

Para garantir uma troca de dados eficiente e evitar falhas críticas (crashes), a camada de interoperabilidade implementa:

- **Mecanismo de Timeout e Cancelamento:** Uso de `CancellationToken` para abortar requisições caso o servidor destino fique indisponível.
- **Tratamento de Exceções Robustas:** Blocos `try-catch` capturando `HttpRequestException` e falhas de parseamento de JSON, convertendo erros sistêmicos em mensagens amigáveis na interface através da propriedade `Mensagem`.

```

try
{
    Ocupado = true;

    /// Verifica no Firebase se existe pelo menos um convite
    /// para este e-mail que ainda não foi usado e não expirou.
    bool temConvite = await _conviteService.ExisteConvitePendenteAsync(Email.Trim());

    if (temConvite)
    {
        /// Como o admin já gerou, precisamos buscar do Firebase
        /// para exibir na janela de simulação. (Crie esse método no seu service se necessário)
        string tokenDoFirebase = await _conviteService.ObterTokenPorEmailAsync(Email.Trim()) ?? "ST-XXXXX";

        /// Passa o token recuperado E o e-mail digitado (Email.Trim())
        var telaSimulada = new ReGraphik.Views.TokenEmailWindow(tokenDoFirebase, Email.Trim());
        telaSimulada.Owner = System.Windows.Application.Current.MainWindow;

        /// Pausa a execução aqui até o usuário fechar a janela de e-mail
        telaSimulada.ShowDialog();

        /// Avança para a etapa 2 na interface (digitar o token)
        SolicitacaoEnviada = true;
        MensagemErroGeral = "E-mail localizado. Digite o token que o Administrador enviou a você.";
    }
    else
    {
        /// E-mail não tem convite - bloqueia sem dar informação útil a invasores
        Mensagem = "Este e-mail não possui um convite de acesso válido. Entre em contato com o Administrador.";
    }
}
}

```

```

/// <exception cref="Exception"></exception>
[HttpGet]
[ProducesResponseType(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
0 referências
public async Task<IActionResult> Get()
{
    try
    {
        var result = await _residuoService.Listar();
        return Ok(result);
    }
    catch (ArgumentException ex)
    {
        // Loga o erro de argumento inválido e retorna um status 400 Bad Request com a mensagem de erro
        _logger.LogWarning(ex, "Requisição inválida ao listar os resíduos");
        return BadRequest("Requisição inválida ao listar os resíduos");
    }
    catch (Exception ex)
    {
        // Loga o erro genérico e retorna um status 500 Internal Server Error com uma mensagem de erro
        _logger.LogError(ex, "Falha ao listar os resíduos");
        return StatusCode(StatusCodes.Status500InternalServerError, "Ocorreu um erro interno ao processar a solicitação.");
    }
}

```

1.4. Persistência em Firebase

ReGraphikFirebase ▾

Realtime Database

✦ Precisa de ajuda com o Realtime Database? Peça ao Gemini!

Dados Regras Backups Uso Extensions

Proteja os recursos do Realtime Database de abusos, como fraude de faturamento ou phishing [Configurar o App Check](#) ✕

https://regraphikfirebase-default-rtdb.firebaseio.com

Anexo:

```

DataCadastro: "2026-06-21T21:02:34.0540979-03:00"
DimensoesCm: 200
DimensoesLm: 150
Especificacao: "Fosco"
Id: "db6d30f2-7674-40ed-a207-a7b7fc2b978d"
IdCard: "#db6d30f2"
IdUsuario: "Id_Do_Usuario_Logado"
Observacao: "Erro no fechamento do arquivo de impressão, lona sem acabamento."
Origem: "Erro de Impressão / Defeito"
Projeto: "Banner Evento Coca-Cola"
Quantidade: 8
Status: "Aguardando CADRI"
TipoResiduo: "Lona"

```

dc9535a1-29aa-4c66-a5cc-1b1b5575169e

dd82ad96-6ccd-40e3-8592-f31827111285

fff9c887-e9f9-4417-9e36-1b32879abd24